

Тема: Цикл while. Обработка цифр числа. Операторы break, continue, else

Цикл **while** в Python используется для **повторения блока кода, пока** заданное условие является истинным (True). Это цикл, который управляется условием, а не количеством повторений.

1. Синтаксис и Структура

Базовая структура цикла while очень проста:

```
# Инициализация переменной (если нужна для условия)
счетчик = 0

while условие_истинно:
    # Блок кода, который будет повторяться
    print("Повторяю")

    # Обязательно: изменение переменной, чтобы условие
    со временем стало ложным
    счетчик += 1

# Код после цикла
print("Цикл завершён")
```

- **while**: Ключевое слово, начинающее цикл.
- **условие_истинно**: Любое выражение, которое можно оценить как True или False (например, `счетчик < 5`).
- **Двоеточие** `:`: Обязательный символ после условия.
- **Отступ (4 пробела)**: Определяет, какой код находится внутри цикла и будет повторяться.

2. Принцип Работы

1. Python **проверяет условие** перед каждым повторением.
2. Если условие **истинно (True)**, выполняется код внутри цикла.
3. После выполнения кода Python снова возвращается к шагу 1 и **снова проверяет условие**.
4. Если условие **ложно (False)**, цикл прекращает работу, и выполнение программы переходит к коду, который идёт после цикла.

3. Ключевые Отличия от for

Характеристика	Цикл while	Цикл for
Управление	Условием. Повторяется, пока условие истинно.	Коллекцией. Повторяется для каждого элемента в последовательности (range, списке и т.д.).
Известно кол-во итераций?	Обычно нет . Используется, когда неизвестно, сколько раз нужно повторять (например, "пока пользователь не введет 'стоп'").	Обычно да . Количество повторений известно заранее.
Завершение	Требует обязательного изменения переменной внутри тела цикла, чтобы условие стало ложным.	Завершается автоматически, когда кончаются элементы в последовательности.

4. Опасность: Бесконечный Цикл

Самая частая ошибка новичков — это создание **бесконечного цикла**.

```
# ПРИМЕР БЕСКОНЕЧНОГО ЦИКЛА (ОШИБКА!)
a = 1
while a < 5:
    print(a)
    # Здесь забыли изменить a! a всегда будет равно 1, и
    # условие всегда True.
    # Цикл никогда не остановится.
```

Чтобы цикл while работал правильно, необходимо гарантировать, что условие рано или поздно станет ложным.

5. Ключевые Операторы внутри Цикла

- **break**: Немедленно **прерывает** выполнение цикла, даже если условие цикла ещё истинно. Полезно для выхода при нахождении нужного результата.
- **continue**: **Пропускает** оставшуюся часть текущей итерации и переходит к проверке условия для следующей итерации. Полезно для пропуска обработки нежелательных данных.

6. Конструкция while...else

Как это работает?

Блок кода в else выполняется **только в том случае**, если цикл while **завершился обычным образом**, то есть **условие цикла стало ложным** (False).

```
счетчик = 0

while счетчик < 3:
    print(f"Итерация {счетчик}")
    счетчик += 1
else:
    # Этот блок выполнится, когда счетчик станет 3 (и
    # условие 'счетчик < 3' станет False)
    print("Цикл завершён штатно.")
```

Когда блок else НЕ выполняется?

Блок else **НЕ** будет выполнен, если цикл был **прерван** командой **break**.

Именно в этом заключается главная **цель** конструкции while...else: узнать, был ли цикл завершен полностью или нет.

Зачем это нужно? (Главный пример)

Конструкция while...else идеально подходит для реализации **поиска**.

Представьте, что вы ищете определенное число в диапазоне.

- Если вы **нашли** число, вы выходите из цикла с помощью **break**. else **НЕ** выполняется.
- Если вы **проверили весь диапазон** и не нашли число, цикл завершается штатно. else **ВЫПОЛНЯЕТСЯ**.

Пример: Поиск числа

Задача: Проверить, есть ли число 15 в диапазоне от 1 до 10.

```
i = 1
```

```

target = 15
found = False

while i <= 10:
    if i == target:
        found = True
        print(f"Число {target} найдено!")
        break # <-- Цикл прерывается! Блок else будет
пропущен.
    i += 1
else:
    # Этот блок выполнится ТОЛЬКО, если не сработал
break.
    print(f"Число {target} не найдено в диапазоне.")

# Результат: Число 15 не найдено в диапазоне. (Выполнился
else)

```

Если бы мы искали число 5:

1. Цикл дойдет до $i = 5$.
2. Сработает `if i == target`, и выполнится `break`.
3. Программа выведет: "Число 5 найдено!"
4. Блок `else` будет **проигнорирован**.

Задачи, Решаемые Циклом `while`

Задача 1: Ввод с проверкой (Валидация)

Уровень: Легкий

Напишите программу, которая запрашивает у пользователя его возраст. Программа должна **продолжать спрашивать**, пока пользователь не введет число от 1 до 100 включительно.

```

# Задача 1: Ввод с проверкой (Валидация)

# Задаем начальное значение возраста, которое
гарантированно не пройдет проверку
age = 0

```

```

# Цикл будет работать, пока возраст НЕ находится в
диапазоне от 1 до 100
while age < 1 or age > 100:
    try:
        # Запрашиваем ввод
        age = int(input("Введите ваш возраст (от 1 до
100): "))
    except ValueError:
        # Обработка случая, если пользователь ввел не
число
        print("Это не число. Попробуйте снова.")
        # Важно: если произошла ошибка, age остается 0
(или любым другим
        # некорректным значением), и цикл повторяется.

# Когда цикл закончится, это означает, что age корректен
print(f"Ваш возраст {age} принят. Спасибо!")

```

Ключевая мысль: Цикл while идеален для **повторного запроса данных**, пока они не удовлетворяют условию.

Задача 2: Обратный отсчет

Уровень: Легкий

Напишите программу, которая ведет обратный отсчет от 10 до 1, после чего выводит "Пуск!".

```

# Задача 2: Обратный отсчет

# Инициализируем счетчик
countdown = 10

# Цикл будет работать, пока счетчик больше или равен 1
while countdown >= 1:

    # Выводим текущее число
    print(countdown)

```

```
# Обязательный шаг: уменьшаем счетчик, чтобы цикл
завершился
countdown = countdown - 1

# Код после цикла
print("Пуск!")
```

Ключевая мысль: В отличие от `for`, где шаг задан в `range()`, в `while` вы должны **сами управлять счетчиком** (`countdown = countdown - 1`).

Задача 3: Вычисление цифр числа

Уровень: Средний

Напишите программу, которая запрашивает у пользователя целое число и **считает, сколько цифр** в этом числе. *(Это та задача, которую мы решали через строки, но математический подход с `while` более элегантен).*

```
# Задача 3: Вычисление цифр числа

# 1. Запрашиваем число
number = int(input("Введите целое число: "))

# Сохраняем оригинальное число для вывода
original_number = number

# 2. Инициализируем счетчик цифр
count = 0

# Цикл должен продолжаться, пока число не станет нулем
while number > 0:

    # 3. Увеличиваем счетчик цифр
    count = count + 1

    # 4. Сокращаем число: делим его нацело на 10
    (отбрасываем последнюю цифру)
```

```
# Пример: 123 // 10 = 12
number = number // 10

# 5. Выводим результат
print(f"В числе {original_number} содержится {count} цифр.")
```

Ключевая мысль: Цикл `while` отлично подходит для **повторяющихся математических операций** (как деление или взятие остатка), которые выполняются до достижения определенного **числового условия** (`number > 0`).

Основные моменты:

1. Цикл `while` выполняет блок кода до тех пор, пока условие остаётся истинным.
2. Обработка цифр числа часто требует использования операций деления на 10 для выделения и удаления цифр.
3. Оператор `break` досрочно завершает цикл, если больше нет смысла продолжать выполнение.
4. Оператор `continue` позволяет пропустить текущую итерацию цикла и перейти к следующей.
5. Оператор `else` используется в циклах и выполняется, если цикл завершён без использования оператора `break`.